📱 HANURI 앱 MVP 기술 스펙 문서 v1.0

문서 버전: MVP v1.0 Approved
작성일: 2026-03-06
작성자: Lead Architect
대상 독자: 개발팀, CTO, 기술 의사결정자

1. 시스템 아키텍처 개요

1.1 전체 시스템 구성도

```
+---------------------+ +------------------------+ +---------------------+ | Mobile Client | | Load Balancer
| | External Services | | (iOS/Android - RN) | <---> | (ALB / API Gateway) | <---> | (OpenAI, Whisper) |
+---------------------+ +------------------------+ +---------------------+ ^ | | WebSocket | REST API v v
+---------------------+ +------------------------+ | Real-time Server | | Backend Server | | (Node.js/Socket.i
| (Node.js/Express) | +---------------------+ +------------------------+ | | +---------------+----------------
+-----------------+-----------------+ | | | +---------------+ +--------------+ +---------------+ | Primary DB
Cache Layer | | Vector DB | | (PostgreSQL) | | (Redis) | | (Pinecone) | +----------------+ +--------------+
                                      +---------------+
```

1.2 아키텍처 결정 사항

영역	결정 사항	선택 근거
아키텍처 패턴	모노리식 (Monolithic)	MVP 단계에서의 빠른 개발 속도, 배포 복잡도 최소화, 팀 규모(소규모) 고려. 추후 트래픽 증가 시 서비스별 분리 예정.
클라우드 인프라	AWS (Amazon Web Services)	안정성, 확장성, 풍부한 관리형 서비스(RDS, ElastiCache), 스타트업 크레딧 활용 가능성.

2. 기술 스택 상세

2.1 프론트엔드 (Mobile)

최종 선택: React Native (Expo)

구분	React Native	Flutter	Native (Swift/Kotlin)
개발 속도	⭐⭐⭐⭐⭐ (Code Push 가능)	⭐⭐⭐⭐	⭐⭐
인력 수급	⭐⭐⭐⭐⭐ (JS 생태계)	⭐⭐⭐	⭐⭐
UI 자유도	⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐
결론	**채택 (MVP 최적)**	보류	보류

- **UI 프레임워크:** React Native Paper / Custom Design System

- **상태 관리:** Zustand (가볍고 직관적) 또는 React Query (서버 상태 동기화)
- **내비게이션:** React Navigation v6
- **애니메이션:** Reanimated 3 + Lottie

2.2 백엔드

- **언어/프레임워크:** Node.js + Express (TypeScript)
  - 프론트엔드와 언어 통일로 생산성 향상, 풍부한 AI 라이브러리 지원.
- **API 패턴:** RESTful API (메인), WebSocket (실시간 대화용)
- **인증:** JWT (Access/Refresh Token), Passport.js (소셜 로그인)
- **API 문서화:** Swagger (OpenAPI 3.0)

2.3 데이터베이스

- **Primary DB:** PostgreSQL (Amazon RDS)
  - 관계형 데이터(유저, 결제, 학습 진도)의 정합성 보장. JSONB 지원으로 유연성 확보.
- **Cache Layer:** Redis (ElastiCache)
  - 세션 관리, API 캐싱, 실시간 대화 컨텍스트 임시 저장.
- **Vector DB:** Pinecone
  - RAG(Retrieval-Augmented Generation) 기반 AI 답변 생성을 위한 임베딩 저장.

2.4 AI/ML 스택

- **LLM (대화/교육):** OpenAI GPT-4o-mini (비용/속도 최적화) + GPT-4o (복잡한 피드백)
- **STT (음성 인식):** OpenAI Whisper API (높은 정확도)
- **TTS (음성 합성):** Google Cloud TTS (다양한 한국어 보이스), ElevenLabs (Pro 기능용 고품질)
- **프롬프트 관리:** 코드 베이스 내 버전 관리 (Git), 환경 변수로 템플릿 관리.

2.5 인프라 & DevOps

- **CI/CD:** GitHub Actions (자동 빌드 및 배포)
- **컨테이너:** Docker (개발/운영 환경 일치)
- **배포 환경:** AWS Elastic Beanstalk 또는 ECS (Fargate)
- **모니터링:** Sentry (에러 추적), CloudWatch (로그/메트릭)

3. 핵심 기능별 기술 명세

3.1 사용자 인증 시스템

- **소셜 로그인:** Google, Apple (iOS 필수) SDK 연동.
- **JWT 전략:**
  - Access Token: 수명 1시간, 메모리 저장.
  - Refresh Token: 수명 2주, HTTP Only Cookie 또는 Secure Storage 저장.

3.2 AI 대화 시스템

실시간성을 보장하기 위해 하이브리드 방식 사용.

- **통신 프로토콜:** WebSocket (Socket.io) 권장. 연결 끊김 시 HTTP 폴백.
- **레이턴시 최적화:** 스트리밍(Streaming) 응답 처리. LLM 토큰이 생성되는 즉시 클라이언트로 전송.
- **Context 관리:** 최근 대화 10턴을 Redis에 저장하고, LLM 요청 시 프롬프트에 포함.

3.3 발음 분석 시스템

```
// 발음 분석 프로세스
1. 클라이언트: 마이크 입력 녹음 (AAC/M4A)
2. 클라이언트 -> 서버: 오디오 파일 업로드 (Multipart)
3. 서버 -> Whisper API: STT 변환 및 타임스탬프, 신뢰도(Confidence) 추출
4. 서버: 기준 스크립트와 STT 결과 비교 (Diff Algorithm)
5. 서버: 유사도 기반 점수 계산 및 피드백 JSON 생성
6. 서버 -> 클라이언트: 분석 결과 반환
```

3.4 결제 시스템

- **iOS:** RevenueCat SDK 사용 (StoreKit 2 복잡도 추상화, 영수증 검증 대행).
- **Android:** RevenueCat SDK 사용 (Google Play Billing).
- **웹훅 처리:** 구독 갱신/취소/환불 이벤트를 백엔드 DB와 동기화.

4. 데이터베이스 스키마 설계 (간략)

4.1 주요 테이블 구조 (ERD)

Users (사용자)

필드명	타입	설명
id	UUID	PK
email	VARCHAR	이메일 (Unique)
native_lang	VARCHAR	모국어 코드 (예: 'en', 'es')
current_level	INT	현재 학습 레벨 (1~12)

Lessons (레슨 메타데이터)

필드명	타입	설명
id	INT	PK
level	INT	레벨
unit	INT	유닛
content_json	JSONB	레슨 상세 내용 (문제, 스크립트)

UserProgress (학습 진도)

필드명	타입	설명

user_id	UUID	FK (Users)
lesson_id	INT	FK (Lessons)
status	ENUM	'locked', 'in_progress', 'completed'
score	INT	획득 점수

5. API 엔드포인트 명세

5.1 인증 (Auth)

- `POST /api/v1/auth/social/google` : 구글 로그인 토큰 검증 및 JWT 발급
- `POST /api/v1/auth/refresh` : Access Token 재발급

5.2 AI 대화 (Conversation)

- `POST /api/v1/ai/chat/start` : 새로운 대화 세션 시작 (시나리오 선택)
- `POST /api/v1/ai/chat/message` : 사용자 메시지 전송 및 AI 응답 요청 (스트리밍)
- `GET /api/v1/ai/chat/{sessionId}/report` : 대화 종료 후 피드백 리포트 조회

5.3 레슨 (Lesson)

- `GET /api/v1/lessons` : 레벨별 레슨 목록 및 잠금 상태 조회
- `GET /api/v1/lessons/{id}` : 특정 레슨 데이터 로드
- `POST /api/v1/lessons/{id}/complete` : 레슨 완료 처리 및 XP 지급

6. 보안 고려사항

- **HTTPS 강제:** 모든 통신은 SSL/TLS 암호화 적용.
- **데이터베이스 암호화:** RDS 저장 데이터 암호화 (At-rest encryption).
- **API Rate Limiting:** IP당 요청 횟수 제한 (DDoS 방지 및 AI API 비용 관리).
- **입력 값 검증:** Joi 또는 Zod 라이브러리를 사용하여 모든 요청 바디 검증.
- **개인정보 최소화:** GDPR 준수를 위해 불필요한 개인정보 수집 지양, 탈퇴 시 즉시 파기.

7. 확장성 및 성능 전략

- **CDN (CloudFront):** 이미지, 오디오 파일 등 정적 자산 캐싱으로 로딩 속도 향상.
- **Connection Pooling:** DB 커넥션 부하 관리.
- **비동기 처리:** 발음 분석, XP 계산 등 시간이 걸리는 작업은 메시지 큐(SQS) 등을 통해 백그라운드 처리 고려 (Phase 2).

8. 개발 로드맵 (MVP)

Phase 1: Core Foundation (Week 1-4)

- 프로젝트 세팅 (RN, Node.js, DB)
- 사용자 인증 및 프로필 관리
- 기본 레슨 UI 및 CMS 구축

Phase 2: AI Implementation (Week 5-8)

- OpenAI API 연동 및 프롬프트 엔지니어링
- 실시간 대화 인터페이스 구현
- Whisper API 기반 발음 기능 구현

Phase 3: Polish & Beta (Week 9-12)

- 결제 시스템 (IAP) 연동
- UI/UX 폴리싱 및 애니메이션 추가
- QA 및 앱스토어 심사 제출

9. 비용 추정 (월간 예상)

초기 사용자 1,000명 기준

항목	서비스	예상 비용
서버/DB	AWS (EC2, RDS, ElastiCache)	$50 ~ $100 (Free Tier 활용 시 절감)
AI API	OpenAI (GPT, Whisper)	$100 ~ $300 (사용량 비례)
TTS	Google Cloud TTS	$20 ~ $50
기타	도메인, 이메일 등	$20
합계		**약 $200 ~ $500**